

YUTA

Workflow v3

Hướng dẫn vận hành Bản A+ cập nhật theo repository

SNAPSHOT / 17 THÁNG 09 NĂM 2026

Bản đọc tham khảo được cập nhật từ nguồn hiện tại. Không tự cấp approval mới, không thay đổi workflow canonical và không mở lại change đã đóng.

Understand broadly → decide narrowly → implement narrowly → verify independently → consolidate knowledge

Phần I: Toàn bộ 18 mục của hướng dẫn canonical hiện tại.

Phần II: Chỉ dẫn bổ sung: engineering skills, Design N/A, external design reference và closure boundaries.

Nguồn gốc: PDF A+ 17 trang ngày 03.09.2026 được giữ nguyên. Metadata APPROVED của nguồn không được dùng để tự gán approval cho bản xuất mới này.

Repository DONE != Pilot authorization != Production authorization

Nội dung

Nội dung	2
Bản cập nhật này thay đổi gì?	3
1. Workflow này là gì	4
2. Vai trò và trách nhiệm	5
3. Tổng quan end-to-end	6
4. Routing cross-module trước một change	7
5. Hướng dẫn vận hành từng bước	8
6. Analysis gate	12
7. Review gates và integrity	12
8. Tasks, phased implementation và Technical Implementation Contract	13
9. VERIFY khác QA	15
External design reference routing	17
10. Gate 3 readiness	17
11. Normative specs và sync	17
12. `\$yuta-run-change`	17
13. `\$yuta-finish-change`	18
14. Knowledge Consolidation	19
15. No-spec path	19
16. Release / Deploy lane	19
17. Ví dụ hằng ngày	20
18. Quick reference	21
19. Engineering skills trong workflow	22
20. Design N/A và no-spec: quyết định riêng biệt	23
21. External design intelligence: tham khảo, không authority	24
22. Closure đúng nghĩa và ngoại lệ có giới hạn	25
23. Operational consistency v3.1	26
24. Nguồn và khả năng truy vết	27

Bản cập nhật này thay đổi gì?

Nội dung	Cách cập nhật
18 mục nền tảng	Lấy trực tiếp từ docs/YUTA_WORKFLOW_V3.md hiện tại, giữ nguyên semantics. Tái bố cục để tránh heading bị rơi cuối trang.
Engineering primitives	Thêm hướng dẫn phân biệt năm kỹ năng hỗ trợ với run/finish orchestration. Chỉ explicit invocation; không cấp authority hoặc rollout.
Design / no-spec	Bổ sung đầy đủ omission criteria, tasks.md evidence và raw graph khác operational readiness.
UI/UX Pro Max	Bổ sung usage classification và advisory-only boundary; không cài đặt/chạy tool hoặc chọn style mặc định.
Closure / Knowledge	Nhấn mạnh hai finish branches, Knowledge sau archive, human approval và không dùng exceptional reconciliation làm workflow mặc định.

Bản vận hành v3.1: Đồng bộ sơ đồ finalization và ba operating prompts theo quyết định hiện tại. Giữ Workflow v3, bốn trạng thái QA và toàn bộ stage; không đổi skills, review packets, archive hoặc Product Knowledge.

Cách dùng: Nếu snapshot này khác repository tại thời điểm thực hiện, dùng nguồn hiện hành và xử lý CONFLICT / NEEDS REVIEW theo Authority Model. Không thực thi câu lệnh trong PDF chỉ vì nó được liệt kê.

1. Workflow này là gì

YUTA Workflow v3 là mô hình vận hành một thay đổi từ ý tưởng đến khi repository được đóng lại có kiểm soát. Mục tiêu là giúp Product, reviewer và engineering cùng nhìn một thay đổi qua các lớp bằng chứng khác nhau, thay vì để một bản mô tả, một lệnh kiểm tra hoặc một trạng thái kỹ thuật tự thay thế quyết định của con người.

Nguyên tắc xuyên suốt là:

Understand broadly
→ decide narrowly
→ implement narrowly
→ verify independently
→ consolidate knowledge

Workflow tách riêng sáu việc:

1. **Product thinking** xác định vấn đề, phạm vi và authority (nguồn có thẩm quyền cho loại câu hỏi đang xét).
2. **Specs** mô tả hành vi quan sát được cần có.
3. **Implementation** hiện thực phạm vi đã duyệt trong code, database hoặc UI.
4. **VERIFY** chứng minh implementation khớp Specs/Design và quy tắc kỹ thuật.
5. **QA** chứng minh hành vi thực tế trong ngữ cảnh người dùng/runtime phù hợp.
6. **Normative sync và Knowledge Consolidation** lần lượt quảng bá yêu cầu đã duyệt vào main specs và đối soát lại knowledge hiện hành.

Release, deploy và Production Readiness không nằm trong định nghĩa repository **DONE**; chúng thuộc một lane vận hành riêng.

2. Vai trò và trách nhiệm

Vai trò / lớp authority	Trách nhiệm chính	Không được tự suy ra
Page Chat	Giữ Product context của một page; thực hiện Cross-Module Impact Check; review các gate page-local.	Không khởi tạo change page-local khi kết quả là CROSS_MODULE hoặc UNCERTAIN.
YUTA Control Tower	Điều phối cross-page/module/runtime; làm rõ owner, authority, shared contract và review routing.	Không thay thế chiều sâu Product context của Page Chat hoặc tự giải quyết blocker bằng assumption.
Codex	Đọc authority, tạo/tiếp tục artifact, implement, kiểm tra, tập hợp evidence và dừng đúng gate.	Không phải Product approver; không tự tạo approval, permission, owner hoặc durable boundary.
OpenSpec	Giữ proposal, analysis, delta specs, design và tasks của một change có cấu trúc.	Change trong openspec/changes/** không phải normative authority.
Product Knowledge	Giải thích WHY, broader WHAT, mục đích module, quan hệ, scope và non-goals.	Không tự chứng minh code hiện tại, deployment hoặc readiness.
Normative main specs	Sau approved sync, định nghĩa yêu cầu hành vi chính xác trong durable boundaries đã chấp nhận.	Không thay thế ADR, security/runtime/data authority, code hoặc bằng chứng production.
Code/tests	Chứng minh repository Implemented State và mức độ test coverage.	Không tự tạo Product approval hoặc chứng minh bản nào đang chạy production.
QA evidence	Chứng minh hành vi người dùng/runtime, responsive, accessibility và visual khi áp dụng.	Screenshot không định nghĩa business rule, permission, schema, owner hoặc lifecycle.
Human reviewer / current user	Ra quyết định tại đúng gate trên đúng artifact/hashes; cấp authorization rõ ràng khi cần.	Một approval không tự mở rộng sang gate, change, path hoặc phiên bản artifact khác.

Các lớp này bổ sung cho nhau. Khi chúng mâu thuẫn, dùng `AUTHORITY_MODEL.md` để phân loại câu hỏi, ghi `CONFLICT` và `NEEDS REVIEW`, rồi dừng ở authority phù hợp; không âm thầm chọn lớp thuận tiện nhất.

3. Tổng quan end-to-end

IDEA

- Discovery/Shaping [conditional]
- Proposal
- Analysis
- Gate 1
- Specs
- Gate 2
- Design [when applicable]
- Sensitive Design Gate [conditional]
- Tasks + Implementation Plan
- Apply
- Verify
- QA
- Gate 3
- Human Approval
- \$yuta-finish-change
- Sync or valid no-spec finalization
- Validate Main Specs when applicable
- Archive
- Knowledge Consolidation
- Done

Discovery/Shaping, Sensitive Design Gate, Gate 2 và normative promotion có điều kiện; các nhánh được giải thích bên dưới. Song song về mặt quản trị, nhưng chỉ bắt đầu sau khi có authorization phù hợp:

RELEASE / DEPLOY / POST-DEPLOY VERIFY / MONITOR / ROLLBACK
= separate operational lane

Design conditionality là chính sách controlled-adaptor của YUTA, không phải native conditional dependency của OpenSpec: raw graph vẫn phụ thuộc Design. Khi không applicable, ghi rõ lý do và authority/evidence trong `tasks.md`, giữ bằng chứng qua resume/review/finalization và không tạo placeholder `design.md`. Chỉ đi tiếp khi các prerequisite/gate khác đã thỏa mãn; raw planning có thể vẫn báo incomplete. Sensitive Design Gate được đánh giá độc lập theo tiêu chí nhạy cảm, không tự phát sinh chỉ vì có Design. `skip_specs: true` không tự bỏ Design.

4. Routing cross-module trước một change

Trước mọi feature/change mới, Page Chat thực hiện CROSS-MODULE IMPACT CHECK theo YUTA_PAGE_CHAT_OPERATING_PROMPT_V3.md và chọn đúng một kết quả:

Kết quả	Cách xử lý
PAGE_LOCAL	Ở lại owning Page Chat. Xác nhận capability, lý do page-local, nhu cầu Discovery/Shaping và OpenSpec readiness.
CROSS_MODULE	Không tạo/tiếp tục local change. Lập handoff theo Control Tower Handoff Template v3.
UNCERTAIN	Xử lý như CROSS_MODULE cho đến khi ownership và scope được làm rõ.

Một thay đổi cần Control Tower khi nó đọc/ghi data của nhiều owner, thay đổi shared permission/security/tenancy, đi qua nhiều runtime, cần provider/legal/ privacy review, chạm durable boundary, hoặc cần contract và QA phối hợp giữa nhiều module. Control Tower dùng YUTA_CONTROL_TOWER_OPERATING_PROMPT_V3.md để chọn một cross-module change, parent coordination với nhiều bounded changes, hoặc trả lại Page Chat nếu thực tế là page-local.

Phân loại PAGE_LOCAL không phải miễn trừ kiểm tra sau đó. OpenSpec analysis là lớp bảo vệ thứ hai: nếu Codex phát hiện cross-module ownership, CONFLICT hoặc requirement-level NEEDS REVIEW, workflow dừng và chuyển coordination sang Control Tower.

5. Hướng dẫn vận hành từng bước

Discovery/Shaping

Main question: Request đã đủ rõ và đúng owner chưa?

Key work: Đọc authority và implementation read-only; làm rõ scope, owner, runtime/data/provider/cross-module uncertainty.

Input: Idea/request và current repository knowledge.

Output: Câu hỏi cần quyết định, bounded request, hoặc kết luận không tạo change.

Stop condition / next gate: Conditional; dừng nếu chưa thể bound an toàn, nếu rõ thì Proposal.

Proposal

Main question: Tại sao cần change và capability nào thay đổi?

Key work: Ghi Why, What Changes, Capabilities, Impact; xác định behavior-changing hay no-spec candidate.

Input: Bounded request.

Output: proposal.md.

Stop condition / next gate: Sang Analysis; chưa phải approval.

Analysis

Main question: Authority và evidence hiện tại có cho phép viết requirements không?

Key work: Phân loại Product Intent, Implemented State, runtime/data/security, lifecycle và unknowns.

Input: Proposal, Product Knowledge, authority/registry, code/tests khi cần.

Output: analysis.md với một kết luận hợp lệ.

Stop condition / next gate: Gate 1; blocker thì BLOCKED_NEEDS_REVIEW.

Gate 1

Main question: Scope và authority có đúng để đi tiếp không?

Key work: Review exact Proposal + Analysis, conflicts, unknowns và hashes.

Input: proposal.md, analysis.md.

Output: docs/reviews/<change>/01-analysis-review.md.

Stop condition / next gate: Dừng chờ human review. Approval mở Specs hoặc no-spec branch.

Specs

Main question: Hệ thống phải có hành vi quan sát được nào?

Key work: Viết delta requirements/scenarios, không đưa implementation plan vào spec.

Input: Approved Gate 1 và artifact instructions.

Output: specs/<capability>/spec.md deltas.

Stop condition / next gate: Strict validation rồi Gate 2.

Gate 2

Main question: Requirements có đúng, đủ và trong approved boundaries không?

Key work: Review exact deltas, scenarios, edge cases, assumptions, validation và hashes.

Input: Delta specs + Gate 1 evidence.

Output: docs/reviews/<change>/02-specs-review.md.

Stop condition / next gate: Dừng chờ human review; bỏ qua chỉ trên approved skip_specs: true.

Design

Main question: Cách hiện thực nào giữ đúng requirements và boundaries?

Key work: Ghi decisions, alternatives, risks, migration/rollback và safe open questions khi design cần thiết.

Input: Approved requirements hoặc approved no-spec path.

Output: design.md khi applicable.

Stop condition / next gate: Sensitive classification; unresolved decision quay lại gate sở hữu nó.

Sensitive Design Gate

Main question: Thiết kế nhạy cảm có được authority phù hợp chấp nhận không?

Key work: Review security/auth, runtime/data owner, migration, payment/fiscal, Personnel/legal/privacy, provider, POS transaction, irreversible hoặc cross-module durable-boundary impact.

Input: Exact Design + earlier approvals.

Output: docs/reviews/<change>/02b-design-review.md.

Stop condition / next gate: Conditional; dừng trước Tasks/Apply cho đến explicit approval.

Tasks

Main question: Làm việc gì, theo dependency nào, và chứng minh completion ra sao?

Key work: Chọn phase cần thiết; thêm checkbox outcomes và embedded TECHNICAL IMPLEMENTATION CONTRACT.

Input: Approved Specs/Design hoặc no-spec path.

Output: tasks.md + implementation plan; UI_AFFECTING/BROWSER_QA_REQUIRED classification.

Stop condition / next gate: Dừng nếu owner/boundary/constraint chưa giải quyết; nếu sẵn sàng thì Apply.

Apply

Main question: Implementation có tạo đúng approved outcome không?

Key work: Thực hiện code/DB/UI/tests theo phases và contracts; đánh dấu task chỉ khi outcome + evidence tồn tại.

Input: Approved planning artifacts, clean scoped baseline.

Output: Scoped implementation + task evidence.

Stop condition / next gate: Durable/Product discovery quay lại gate; technical defect trong scope có thể sửa rồi tiếp tục.

Verify

Main question: Repository implementation có khớp Specs/Design và technical authorities không?

Key work: Mapping requirements, tests, typecheck/build, architecture/security, migration/schema, diff và Compliance Matrix.

Input: Implementation + approved planning.

Output: TECHNICAL IMPLEMENTATION COMPLIANCE và VERIFY result.

Stop condition / next gate: Không PASS thì sửa trong approved behavior hoặc quay lại gate phù hợp.

QA

Main question: Hành vi có hoạt động đúng trong user/runtime context không?

Key work: Chạy Browser QA hoặc non-browser QA phù hợp; lưu report/evidence.

Input: Verified implementation và realistic environment/data.

Output: PASS, FAIL, BLOCKED_BY_ENVIRONMENT, hoặc NOT_APPLICABLE.

Stop condition / next gate: Required QA chưa PASS thì chưa được tạo ready Gate 3.

Gate 3

Main question: Exact implementation đã đủ an toàn để final approval chưa?

Key work: Independent review planning hashes, attributed diff, Technical Compliance, VERIFY, QA, deviations và lifecycle truth.

Input: Toàn bộ current evidence.

Output: docs/reviews/<change>/03-final-review.md.

Stop condition / next gate: Dừng chờ explicit Gate 3 approval và explicit sync/archive authorization.

Sync

Main question: Approved deltas nào được mechanical promotion?

Key work: Recheck hashes; sync đúng existingOutputPaths; review exact main-spec diff.

Input: Approved Gate 3 + explicit authorization.

Output: Updated normative main specs, hoặc không có promotion trên no-spec path.

Stop condition / next gate: Partial/unexpected sync dừng và rollback có ghi nhận; thành công thì Validate.

Validate

Main question: Main-spec result có hợp lệ và đúng approved delta không?

Key work: Strict main-spec validation và diff review.

Input: Synced main specs.

Output: Validated main-spec state.

Stop condition / next gate: Failure: không archive như thành công; success: Archive.

Archive

Main question: Change history có thể đóng an toàn chưa?

Key work: Reconfirm task/artifact completion và sync state; archive đồng bộ.

Input: Validated sync hoặc valid no-spec finalization.

Output: Recorded archive.

Stop condition / next gate: Archive thành công mới sang Knowledge Consolidation.

Knowledge Consolidation

Main question: Current knowledge có cần cập nhật từ completed evidence không?

Key work: Scan đúng các knowledge authorities có thể bị ảnh hưởng; không tự promote status.

Input: Archived completed change.

Output: NO_UPDATE_REQUIRED hoặc UPDATE_REQUIRED + review packet.

Stop condition / next gate: Update required thì dừng chờ Knowledge Review; no update thì Done.

Done

Main question: Repository workflow đã đóng trung thực chưa?

Key work: Ghi workflow outcome và `RELEASE_FOLLOW_UP`.

Input: Completed consolidation path.

Output: Workflow status: DONE.

Stop condition / next gate: Chỉ chuyển sang release/deploy lane nếu được yêu cầu và authorized.

6. Analysis gate

YUTA thêm analysis để ngăn việc một request đi thẳng từ ý tưởng sang requirements bằng assumption. Analysis phải kiểm tra:

- Product Knowledge cho broader intent và scope;
- Authority Model để chọn nguồn kiểm soát theo loại câu hỏi;
- Module Registry và Lifecycle Status Model cho owner, relationship và trạng thái bounded hiện tại;
- code/tests/contracts/executable schemas chỉ để xác minh Implemented State;
- runtime, data ownership, security/authorization và production evidence theo đúng authority;
- mọi CONFLICT và NEEDS REVIEW còn tồn tại.

Kết luận Analysis hợp lệ **chính xác** là:

READY_FOR_SPECS
BLOCKED_NEEDS_REVIEW
NO_SPEC_BEHAVIOR_CHANGE

CONFLICT là một finding (phát hiện), không phải Analysis conclusion. Nếu conflict ảnh hưởng requirement readiness, conclusion phải là **BLOCKED_NEEDS_REVIEW**; approval chung chung không giải quyết được blocker.

7. Review gates và integrity

Gate	Reviewer đang quyết định gì?	Review packet
Gate 1 - Product / Authority Review	Proposal/Analysis có đúng scope, owner, authority và đủ rõ để viết Specs hoặc đi no-spec không?	01-analysis-review.md
Gate 2 - Requirements Review	Exact requirements/scenarios có đúng approved behavior, không chứa hidden assumption và strict-valid không?	02-specs-review.md
Sensitive Design Gate	Thiết kế nhạy cảm có giữ đúng durable, security, data/runtime, legal/provider và rollback boundaries không?	02b-design-review.md
Gate 3 - Final Independent Review	Exact implementation/evidence có đủ để approve finalization và cho phép sync/archive không?	03-final-review.md

PASS, test xanh, file tồn tại, commit/PR approval hoặc recommendation của Codex không thay thế human decision. Mỗi approval bị ràng buộc bởi exact path set và SHA-256 của bytes đã review. Khi thêm, xóa, đổi tên hoặc thay bytes của artifact/evidence, packet liên quan trở thành **INVALIDATED_BY_ARTIFACT_CHANGE** và phải được review lại.

Các trạng thái review packet là **AWAITING_HUMAN_REVIEW**, **APPROVED**, **CHANGES_REQUESTED**, và **INVALIDATED_BY_ARTIFACT_CHANGE**. Approval phải đến từ current user, ghi rõ change/gate đang duyệt; Gate 3 còn cần authorization rõ ràng cho sync và archive.

8. Tasks, phased implementation và Technical Implementation Contract

Các phase sau là nhãn lập kế hoạch tùy chọn, không phải chuỗi stage bắt buộc:

Phase tùy chọn	Khi nào cần	Nội dung cần làm rõ khi liên quan
Foundation / Data	Có thay đổi nền tảng dữ liệu hoặc persistence.	Owner/canonical owner, schema/migration khi áp dụng, tenant/isolation concerns và required evidence/checks từ phase contract.
Service / Domain	Có thay đổi nghiệp vụ hoặc service.	Domain behavior, trusted boundaries, validation, authorization khi áp dụng và targeted tests/checks.
UI / Components	Có thay đổi cấu trúc hoặc presentation UI.	Component ownership, Server/Client boundary, reuse approved UI patterns và evidence rằng approved UI outcome đã được hiện thực.
Interaction / States	Có thay đổi tương tác hoặc trạng thái.	Edit/read-only, loading/error/pending/success/recovery, keyboard/basic accessibility và relevant state behavior.
Integration / Regression	Cần chứng minh phối hợp hoặc tránh regression.	Relevant integration/end-to-end/regression checks và evidence cho affected behavior.

Các concerns trong bảng chỉ áp dụng khi liên quan đến change, không phải requirements bắt buộc cho mọi phase. Quy tắc chọn phase:

- Chỉ chọn phase change thực sự cần.
- Sắp các phase được chọn theo dependency.
- Không buộc change nhỏ đi qua đủ năm phase.
- Một phase có thể chứa nhiều task cụ thể.
- Không tạo phase rỗng.

Technical Implementation Contract

Trong mỗi phase được chọn của `tasks.md`, nhúng một **TECHNICAL IMPLEMENTATION CONTRACT**. Đây là một phần của Tasks / Implementation Plan, không phải OpenSpec artifact mới. Contract ghi rõ:

- affected boundary;
- canonical owner;
- applicable scoped authorities, gồm root/nearest scoped `AGENTS.md` và các repository authority áp dụng;
- applicable constraints;
- intended files/packages;
- required targeted checks;
- completion evidence.

Code, DB, UI và test thật được thay đổi ở **APPLY**, không phải khi viết contract. Không bắt đầu **APPLY** khi technical owner, boundary, authority hoặc constraint cần thiết còn chưa giải quyết. Classification QA được ghi trong Tasks / Implementation Plan trước **APPLY** (xem mục 9.A).

Vòng thực hiện APPLY

Trong mỗi phase được chọn:

Đọc task + contract + authority được dẫn chiếu
→ implement approved scope
→ chạy targeted checks khi phù hợp trước khi đi tiếp
→ đánh giá mọi contract item áp dụng
→ ghi evidence
→ chỉ đánh dấu complete khi implementation outcome VÀ compliance evidence tồn tại

Code đã thay đổi chưa đủ để complete task. Một checkbox chỉ complete khi implementation outcome **và** compliance evidence cùng tồn tại; trước khi hoàn tất phase phải đánh giá mọi contract item áp dụng. Vòng này không thêm human approval sau mỗi phase. UI checks trong APPLY không thay thế Browser QA sau VERIFY.

Technical defect trong approved behavior

- Có thể sửa implementation defect trong approved scope.
- Chạy lại relevant targeted checks.
- Tiếp tục khi contract và evidence đã được đáp ứng; không tự tạo Product decision mới.

Discovery cần thay đổi quyết định

Nếu implementation cho thấy cần thay đổi:

- Product behavior;
- requirement;
- permission/security;
- canonical owner;
- API/contract;
- runtime/data/durable boundary;

thì **STOP** và quay lại Product/authority/Design gate phù hợp. Không làm yếu hoặc âm thầm viết lại Specs/Design để khớp code; không bỏ qua technical rule để hoàn tất phase.

9. VERIFY khác QA

VERIFY - technical correctness

Checks trong APPLY hỗ trợ triển khai từng task/phase. VERIFY đối chiếu toàn bộ implementation trong scope với approved planning và technical authorities; QA kiểm tra hành vi trong user/runtime context. Ba việc này không thay thế nhau.

VERIFY trả lời "implementation trong repository có khớp approved Specs/Design và technical authorities không?". Evidence có thể gồm:

- Specs/Design → code/test mapping;
- targeted và broader tests;
- typecheck, build và strict OpenSpec validation;
- architecture, security, schema và migration checks;
- scoped diff, deviations và blockers.

TECHNICAL COMPLIANCE MATRIX nằm trong VERIFY evidence hiện có, không phải artifact mới. Mỗi constraint áp dụng được truy vết theo:

```
constraint → authoritative source → affected implementation  
→ check/evidence → PASS | FAIL
```

Matrix chỉ chứa constraints áp dụng và traceability tới các phase thực sự dùng; không thêm dòng rỗng cho phase không cần. Chỉ ghi TECHNICAL IMPLEMENTATION COMPLIANCE: PASS khi mọi dòng áp dụng đều PASS. VERIFY: PASS còn đòi hỏi implementation khớp approved Specs/Design và không còn deviation/blocker ngăn technical verification. Nếu không đạt, ghi FAIL hoặc blocker cụ thể; sửa trong approved behavior rồi chạy lại, hoặc quay về gate phù hợp nếu cần quyết định ngoài scope. Không tạo status mới cho finding.

VERIFY không chứng minh browser UX, visual/responsive correctness, deployment, environment enablement hoặc Production Readiness.

QA - user/runtime correctness

QA trả lời "hành vi có hoạt động đúng trong ngữ cảnh người dùng/runtime không?". Các nhóm A-E dưới đây chỉ tổ chức phần giải thích, không phải QA phases, states hoặc gates mới. Chi tiết dùng YUTA_QA_PROTOCOL.md.

A. QA scope classification

Tasks / Implementation Plan ghi trước APPLY:

```
UI_AFFECTING: YES | NO  
BROWSER_QA_REQUIRED: YES | NO
```

Thay đổi UI-visible, interaction, responsive, role/edit/read-only state hoặc loading/error/success presentation đặt cả hai giá trị là YES và bắt buộc Browser QA. Non-UI dùng non-browser QA khi có user/runtime QA dimension; NOT_APPLICABLE chỉ khi thật sự không có dimension đó, không hợp lệ với UI_AFFECTING: YES. Backend/data-only không cần Browser QA vô nghĩa; correctness của schema, migration, repository, tenancy, authorization và integration thuộc VERIFY.

B. Environment/data preparation

QA dùng verified implementation. Browser QA chạy real/local route với dữ liệu realistic an toàn gần thực tế nhất, giữ đúng authorization, tenancy, persistence và runtime boundaries. Không thay integrated page bằng fixtures để làm QA PASS.

Nếu môi trường/dependency không sẵn sàng, chỉ thử safe bounded recovery theo repository; ghi exact commands, failures và hành động môi trường cần thiết. Không che blocker hoặc tự chuyển thành PASS (xem nhóm E).

C. Applicable behavior checks

Kiểm tra các trường hợp áp dụng cho change, không ép mọi scenario lên mọi change nhưng không bỏ scenario liên quan:

- primary happy path và meaningful before/after state;
- role/permission, editable và read-only states;
- loading, error, saving, success và recovery presentation;
- keyboard operation và basic accessibility;
- overflow/clipping, responsive behavior và regression quanh vùng thay đổi.

Viewport của page pack được ưu tiên; nếu không có, tối thiểu desktop 1366x768 và mobile 390x844, thêm intermediate/tablet khi layout hoặc target yêu cầu. Không tạo screenshot trùng lặp không chứng minh layout/state khác biệt.

D. Evidence

UI-affecting change lưu evidence tại:

```
docs/reviews/<change-name>/qa/
□□□ QA_REPORT.md
□□□ screenshot-manifest.md
□□□ *.png
```

QA_REPORT.md ghi scope/classification, route, data/test setup, roles/states, viewports, scenarios, accessibility, visual/responsive/regression findings, limitations và screenshot evidence. Mỗi screenshot phải đến từ actual Browser QA; manifest ghi repository-relative path, viewport, role/state, scenario cùng lowercase SHA-256 của exact file bytes. Gate 3 liên kết và hash report, manifest và screenshots theo review integrity ở mục 7.

Screenshot không thay thế kiểm tra tương tác và không định nghĩa Product behavior, permission, schema, persistence, ownership hoặc lifecycle.

E. Status and next action

QA status giữ nguyên đúng bốn giá trị:

Status	Ý nghĩa và hành động tiếp theo
PASS	Mọi required QA và evidence đã hoàn tất; vẫn phải thỏa các điều kiện Gate 3 khác.
FAIL	Có lỗi behavior/visual/responsive/accessibility áp dụng; ghi finding và xử lý theo phạm vi bên dưới.
BLOCKED_BY_ENVIRONMENT	Required QA không chạy được sau safe bounded recovery vì environment/dependency; ghi blocker và dừng với hành động môi trường cần thiết.
NOT_APPLICABLE	Không có user/runtime QA dimension; không hợp lệ với UI-affecting change.

Với QA FAIL:

- Implementation defect trong approved behavior → sửa trong scope → rerun VERIFY và QA.
- Cần thay đổi behavior/requirement/permission/owner/API hoặc durable boundary → **STOP** và quay lại authority/gate phù hợp, không tự đổi requirements để làm QA PASS.

External design reference routing

Trong Analysis/plan và TECHNICAL VERIFY hiện có, dùng External Design Intelligence để ghi usage, provenance và disposition. Không thêm phase; Gate 3 vẫn đánh giá riêng Technical Implementation Compliance, VERIFY và QA. Không sửa approved Analysis bytes.

10. Gate 3 readiness

Gate 3 chỉ ready khi:

```
TECHNICAL IMPLEMENTATION COMPLIANCE = PASS
VERIFY = PASS
required QA = PASS
```

Với non-UI change, applicable non-browser QA phải PASS; QA: NOT_APPLICABLE chỉ hợp lệ khi thật sự không có user/runtime QA dimension. FAIL, BLOCKED_BY_ENVIRONMENT, thiếu role/state, responsive coverage hoặc evidence bắt buộc đều ngăn ready Gate 3. Dùng phạm vi QA và evidence ở mục 9; readiness không thay thế human approval hoặc sync/archive authorization ở mục 7.

11. Normative specs và sync

```
approval permits sync
sync performs mechanical promotion
```

openspec/changes/** luôn là proposed/in-progress và non-normative. Nội dung main spec chỉ trở thành normative cho precise observable behavior sau khi exact delta qua accountable approval gate, current user cấp explicit sync authorization, sync thành công, và resulting main specs qua validation + diff review.

Sync không tạo Product approval, không thay accepted durable boundaries và không tự cập nhật lifecycle. Main specs cũng không chứng minh implementation, deployment, database shape hoặc Production Readiness. Xem OPENSPEC_YUTA_NORMATIVITY_POLICY_REVIEW.md cho conflict, rollback và modification policy chi tiết.

12. \$yuta-run-change

Đây là workflow hằng ngày để bắt đầu một bounded change, resume một change hoặc adopt một in-flight change. Nó luôn tìm gate sớm nhất còn thiếu, chưa duyệt, changes-requested hoặc invalidated; artifacts về sau không được bypass gate trước. Nó tự làm việc giữa các gate, dừng ở human review kế tiếp, và **không bao giờ sync hoặc archive**.

Ví dụ bắt đầu:

```
$yuta-run-change <mô-tả bounded change>
```

Ví dụ resume sau một approval cụ thể:

```
$yuta-run-change <change-name>
Analysis review approved. Continue.
```

```
$yuta-run-change <change-name>
Specs review approved. Continue.
```

Với in-flight change, workflow giữ nguyên bytes hiện có, tạo packet cho gate sớm nhất cần review và không tự "sửa cho đẹp" artifact đã tồn tại.

13. \$yuta-finish-change

Hai mode có precondition và integrity scope riêng, không được trộn.

Branch A - Active finalization

Gate 3 approval + explicit sync/archive authorization
→ integrity recheck
→ sync (hoặc no normative promotion trên valid no-spec path)
→ validate
→ archive
→ knowledge scan

Lệnh/resume intent điển hình:

```
$yuta-finish-change <change-name>  
Final review approved. I authorize spec sync and archive.
```

Workflow recheck planning artifacts, attributed implementation diff, VERIFY, Technical Compliance, earlier gates và applicable QA hashes trước khi ghi approval. Drift dừng finalization và trả change về \$yuta-run-change.

Branch B - Archived Knowledge Review resume

Gate 3 already approved + archive completed
→ approve exact knowledge diff
→ recheck target path/hashes + proposed-diff hash
→ apply exact approved documentation update
→ validate docs
→ DONE

Ví dụ intent:

```
$yuta-finish-change <change-name>  
Knowledge consolidation review approved. Apply and close.
```

Branch B không tái tạo active change, không yêu cầu Gate 3 approval mới, không rerun sync/archive và không dùng lại authorization cũ để cấp quyền sửa docs.

14. Knowledge Consolidation

Archive chưa phải bước cuối. Sau archive, scan có giới hạn:

- Page Product Knowledge và page-pack/as-built;
- Module Product Knowledge;
- docs/PRODUCT_KNOWLEDGE.md routing;
- Module Registry và lifecycle/current-state;
- ADR/durable decisions;
- CURRENT_STATE.md khi broad summary thay đổi thực chất;
- NEEDS REVIEW thật sự được completed change giải quyết;
- limitation hoặc future work mới phát hiện.

NO_UPDATE_REQUIRED

- record reason + sources inspected
- DONE

UPDATE_REQUIRED

- docs/reviews/<change>/04-knowledge-consolidation-review.md
- human review
- recheck exact hashes
- apply approved docs diff
- validate
- DONE

Knowledge Consolidation không tự approve Product Decision, đổi durable boundary/owner/permission/API, promote lifecycle/readiness, rewrite normative specs hoặc giải quyết NEEDS REVIEW bằng assumption. Chi tiết nằm trong YUTA_KNOWLEDGE_CONSOLIDATION_PROTOCOL.md.

15. No-spec path

skip_specs: true chỉ dùng khi thật sự không có spec-level behavior change, ví dụ pure refactor, tooling hoặc docs. Không tạo requirement giả để vượt validation.

Gate 1 approved

- no Gate 2
- no normative spec promotion
- continue applicable Design/Tasks/Apply/Verify/QA
- Gate 3
- Human Approval
- \$yuta-finish-change
- valid no-spec finalization
- Archive
- Knowledge Consolidation
- DONE

Gate 3 vẫn cần human approval và explicit finalization authorization; mọi Technical Compliance, VERIFY và applicable QA rule vẫn áp dụng. Nếu delta specs tồn tại trên một claimed no-spec path, đó là conflict và workflow phải dừng.

16. Release / Deploy lane

Sau repository DONE, ghi đúng một giá trị:

RELEASE_FOLLOW_UP: NOT_REQUIRED | REQUIRED | UNKNOWN

Nếu required, xác định runtime/environment, deployment authorization, migration hoặc provider/readiness evidence, post-deploy verification, monitoring và rollback. Đây là operational work riêng; Workflow v3 không tự deploy và không dùng QA repository để tuyên bố production enabled.

17. Ví dụ hằng ngày

Example A - small page-local UI change

Page Chat xác nhận PAGE_LOCAL; nếu scope đã rõ thì bỏ Discovery/Shaping. Change đi qua Proposal → Analysis → Gate 1 → Specs → Gate 2; Design chỉ tạo khi applicable và không có Sensitive Design Gate nếu không chạm boundary nhạy cảm. Tasks chọn UI / Components, Interaction / States và regression cần thiết. Sau Apply, VERIFY chứng minh mapping/code/tests; Browser QA chạy real route ở viewport theo page pack, kiểm tra relevant state/accessibility và lưu screenshot hashes. Gate 3 chỉ ready khi Technical Compliance, VERIFY và QA đều PASS.

Example B - backend/data-only change

Sau các Product/requirements/design gate applicable, Tasks có thể dùng Foundation / Data, Service / Domain và Integration / Regression. VERIFY phải chứng minh schema/migration, repository behavior, tenant isolation, authorization, integration và rollback concerns. Không ép Browser QA nếu không có UI; QA: NOT_APPLICABLE chỉ dùng khi cũng không có distinct runtime/user QA dimension. Gate 3 vẫn review exact technical evidence và diff.

Example C - cross-module feature

Page Chat thực hiện impact check và trả CROSS_MODULE hoặc UNCERTAIN, lập handoff rồi dùng local change. Control Tower map owning/consumer capabilities, data owners, authority, shared contract và quyết định một coordinated change hay nhiều bounded changes. Chỉ sau khi Product/durable blockers được giải quyết, Control Tower mới cấp bounded \$yuta-run-change request và routing cho các gate/QA liên quan.

18. Quick reference

CORE

IDEA → [Discovery] → Proposal → Analysis → Gate 1
 → Specs → Gate 2
 → Design [when applicable]
 → Sensitive Design Gate [conditional]
 → Tasks → Apply → Verify → QA
 → Gate 3 → Human Approval → \$yuta-finish-change
 → Sync or valid no-spec finalization
 → Validate Main Specs when applicable → Archive
 → Knowledge Consolidation → DONE

PACKETS

01-analysis-review.md
 02-specs-review.md
 02b-design-review.md conditional
 qa/QA_REPORT.md required for UI-affecting
 qa/screenshot-manifest.md required for UI-affecting
 03-final-review.md
 04-knowledge-consolidation-review.md conditional after archive

COMMANDS

\$yuta-run-change = start/resume/adopt; stop at next human gate
 \$yuta-finish-change = authorized finalization OR archived knowledge resume

ESCALATION

PAGE_LOCAL → owning Page Chat
 CROSS_MODULE / UNCERTAIN → YUTA Control Tower
 Codex finding CONFLICT / requirement-level NEEDS REVIEW → stop and route

QA

UI-affecting → Browser QA + responsive/state/accessibility + hashed screenshots
 Backend/data-only → technical correctness in VERIFY; no meaningless Browser QA

KNOWLEDGE

Archive → scan → NO_UPDATE_REQUIRED or reviewed exact knowledge diff → DONE

Các bất đẳng thức phải giữ nguyên:

APPROVED != IMPLEMENTED
 IMPLEMENTED != PRODUCTION_ENABLED
 VERIFY != QA
 QA PASS != PRODUCTION_READINESS
 ARCHIVE != KNOWLEDGE_CURRENT
 SYNC != PRODUCT_APPROVAL
 WORKFLOW_PROGRESS != LIFECYCLE_PROMOTION

19. Engineering skills trong workflow

Đây là chỉ dẫn từ các skill contracts hiện tại, không phải phase hoặc gate mới. Năm primitives đã có trong repository nhưng việc tài liệu nhắc tên không tự kích hoạt chúng. Chỉ áp dụng khi current user gọi rõ đúng skill và scope/stage/authority đã đủ.

Skill	Dùng khi nào	Không được suy ra
yuta-research	Nghiên cứu câu hỏi có nguồn; Discovery/planning hoặc hỗ trợ được duyệt.	Không tự implement, ghi report hoặc approve Product.
yuta-diagnose-bug	Chẩn đoán có evidence; mặc định read-only. Reproduction cần execution authority.	Diagnosis không phải quyền fix; instrumentation cần write authority.
yuta-slice-tasks	Chia Tasks / Implementation Plan / TIC sau applicable planning gates.	Dependency-ready không phải execution-authorized; không tự mở Apply.
yuta-tdd	Test-first trong AUTHORIZED_APPLY_ONLY, có approved task/behavior và exact scope.	Skill invocation không cấp Apply; test xanh không approve Gate 3.
yuta-code-review	Review độc lập STANDARDS, SPEC, TIC, VERIFY, QA trên delivery đúng scope.	Không auto-fix, gộp các trục thành một điểm, hoặc tự approve Gate 3/Sync/Archive.

Chọn orchestration hay primitive?

yuta-run-change điều phối change đến human gate kế tiếp. yuta-finish-change quản lý authorized finalization hoặc archived Knowledge resume. Năm primitives chỉ giúp một công việc cụ thể trong scope đang được cho phép; không thay hai workflow này.

Discovery/listing/đọc SKILL.md không phải invocation. Invocation không cấp WRITE, EXECUTE, DELETE, install, Gate approval, pilot hoặc production. Đây là INSTRUCTION_LEVEL_ENGINEERING_PRIMITIVES, không phải SECURITY_ENFORCEMENT_MECHANISMS.

Evidence và giới hạn

Giữ riêng static contract evidence, behavioral evaluation và security claims. Kiểm tra tác động thật của command, kể cả setup/cleanup/subprocess; nhấn "check" hoặc "read-only" không tự chứng minh quyền execution. Bảo vệ dirty work và exact path/preimage; không tự stage, commit, rebaseline hoặc sửa integration files.

Theo quyết định post-closure hiện có: A05/A07/A16/A19 giữ KNOWN_EVIDENCE_LIMITATION; A20 giữ deferred security claim. Đây là disposition của integration cụ thể, không thêm QA status vào protocol chung. Việc cập nhật PDF không mở lại integration hoặc chạy lại evaluation. Pilot và Production vẫn NOT_AUTHORIZED.

Nguồn: năm .agents/skills/yuta-*/SKILL.md nêu trên; _engineering-primitives/AUTHORITY-PREFLIGHT.md; scripts/engineering-skills/README.md. Hold/limitations được giữ theo quyết định Control Tower, không được dùng như permission cho task mới.

20. Design N/A và no-spec: quyết định riêng biệt

skip_specs: true không tự bỏ Design. Đánh giá Design trên approved scope: architecture/cross-cutting; data/runtime ownership; security/authorization; migration/destructive data; significant dependency/provider; performance/operational complexity; unresolved technical decisions.

Nếu không tiêu chí nào áp dụng, ghi block sau trong Tasks khi được phép tạo. Các placeholder phải được thay bằng findings thật trước Apply; không tạo design.md giả.

DESIGN APPLICABILITY

Status: NOT_APPLICABLE

Reason: bounded rationale

Applicability criteria checked: findings cho cả bảy tiêu chí

Authority / evidence: exact sources + approved scope/spec references

Expected artifact state: design.md intentionally absent

Controlled adapter chỉ cho tiếp tục khi omitted Design là dependency thiếu duy nhất và mọi prerequisite/human gate khác đã thỏa. Raw OpenSpec có thể giữ Design ready, Tasks blocked trước khi tạo, hoặc isPlanningComplete false sau đó. Luôn báo RAW OPENSPEC STATUS riêng với YUTA OPERATIONAL READINESS.

Resume phải kiểm tra omission evidence, reviewed Tasks hash và expected absence. Scope/rationale drift hoặc Design xuất hiện phải xử lý invalidation; không backfill âm thầm. Sensitive Design Gate đánh giá riêng, không được phân loại N/A để tránh review nhạy cảm.

Ở finish, chỉ warning vì exact reviewed Design omission mới được chấp nhận theo policy hiện hành. Incomplete task hoặc warning khác vẫn chặn. No-spec không có normative promotion, nhưng còn đủ applicable Tasks/VERIFY/QA/Gate 3/Archive/Knowledge.

Nguồn: docs/YUTA_AUTOMATED_CHANGE_WORKFLOW.md, "Conditional Design and persisted omission"; run skill State 3; finish skill "Branch A Design omission integrity".

21. External design intelligence: tham khảo, không authority

UI/UX Pro Max là DESIGN_REFERENCE. Nếu external advice mâu thuẫn YUTA, giữ YUTA authority; nếu các YUTA authorities mâu thuẫn, STOP / CONFLICT / NEEDS REVIEW. Không tự thay Product scope, design system, font, tokens hoặc application rules.

Trong existing Analysis/Gate 1, rồi carry forward vào Tasks và TECHNICAL VERIFY, ghi:

UI_UX_PRO_MAX_USAGE: REQUIRED | OPTIONAL | NOT_APPLICABLE

Reason: scope-specific rationale

Scope: affected question, app/page/change

Decision source: owning reviewed decision/reference

REQUIRED đòi actual accepted/verified tool evidence; thiếu tool hoặc drift là blocker, không auto-install hoặc hạ classification. OPTIONAL có thể NOT USED nhưng phải ghi rõ. NOT_APPLICABLE phải có lý do. Tool không tự đánh giá integration của chính nó.

External findings không thay Technical Compliance, VERIFY hoặc QA; REQUIRED tool failure không được ghi PASS. Discovery/reading không cấp installation hoặc execution. Chỉ dùng supported query surface khi được phép. Không MASTER.md, persistent design system, automatic updates hoặc numeric dials. Không có quyết định style toàn project hay từng app chỉ từ variance/motion/density.

Nguồn: docs/ui/EXTERNAL_DESIGN_INTELLIGENCE.md. Tài liệu này không xác nhận current installed/runtime readiness và không triển khai tool.

22. Closure đúng nghĩa và ngoại lệ có giới hạn

Active finalization

Gate 3 approval và explicit sync/archive authorization phải đúng change/evidence. Recheck earlier gates, planning, attributed diff, VERIFY/Compliance và applicable QA. Nếu behavior-changing: sync exact delta, review diff, strict main-spec validation rồi archive. Nếu valid no-spec: không tạo/sync main spec, tiếp tục khi các điều kiện còn lại đạt.

Archived Knowledge resume

Chỉ sau archive thành công mới scan Knowledge. NO_UPDATE_REQUIRED cần lý do và exact sources inspected. UPDATE_REQUIRED cần exact proposed diff, current target hashes và human review trước apply. Branch B chỉ kiểm tra Knowledge approval boundary; không recreate active change hoặc mở lại Gate 3/Sync/Archive.

Không dùng lịch sử riêng làm policy chung

Present-day lifecycle reconciliation là record mới theo authority riêng, không dựng lại original planning và không biến prior implementation thành task của record mới. Fast-track hoặc accepted limitations của một change không tự áp dụng cho change khác. Không suy ra cơ chế recovery/waiver chung từ một trường hợp đã đóng.

Các closure actions chỉ complete sau thực thi; không đánh dấu archive/Knowledge trước để làm đầy checklist. WORKFLOW DONE không cấp pilot, production hoặc quyền rerun. Khi có post-closure hold, chờ current-user request mới đúng scope.

Nguồn: finish skill và Knowledge Consolidation protocol. Phần reconciliation là lưu ý ranh giới từ quyết định Control Tower cho trường hợp đã đóng, không sửa protocol repository.

23. Operational consistency v3.1

Intake trước khi tạo hoặc tiếp tục change

Ghi Existing OpenSpec change: YES / NO / UNKNOWN; exact active/archive path; current workflow state; implementation, VERIFY, QA, known limitations và historical FAIL/BLOCKED. UNKNOWN cần discovery có giới hạn, không phải quyền tạo mới. Chat mới không có nghĩa là change mới.

Finalization không bỏ qua orchestrator

Gate 3 → Human Approval → \$yuta-finish-change
→ Sync or valid no-spec finalization
→ Validate Main Specs when applicable → Archive
→ Knowledge Consolidation → DONE

Đây là cách biểu diễn chính xác các stage đã có, không thêm stage. Sync/archive cần authorization rõ ràng. Archived Knowledge resume dùng riêng Knowledge boundary, không mở lại original closure.

QA và limitations là hai trường riêng

QA chỉ dùng PASS / FAIL / BLOCKED_BY_ENVIRONMENT / NOT_APPLICABLE. KNOWN_EVIDENCE_LIMITATIONS là trường riêng, không tương đương PASS hoặc tự miễn mandatory evidence. Gate 3 có thể tổng hợp limitations ở mức review/change mà không đổi QA Protocol. Không che implementation/behavioral failure đã xác lập.

Anti-loop và lịch sử trung thực

Giới hạn attribution → correction → revalidation theo câu hỏi và stop condition được duyệt. Established failure có thể cần correction được cấp quyền. Tooling/runtime/evaluator limitation chỉ được tiếp tục khi approved criteria cho phép; nếu thiếu required evidence thì dừng, không xây generic evidence infrastructure để đổi mọi limitation thành PASS.

Không dựng lại original artifacts chưa từng tồn tại, không đổi historical FAIL/BLOCKED, không tạo change chỉ để tooling xanh. Missing lifecycle inputs phải escalate FINISH_CHANGE_BLOCKED / LIFECYCLE_RECONCILIATION_REQUIRED. Present-day reconciliation cần governance authorization và archive không giả làm original implementation history.

Nguồn: ba operating prompts v3.1 và các protocol hiện tại, có hash bên dưới. Tên file V3 được giữ ổn định; v3.1 là cập nhật vận hành, không phải workflow architecture mới.

24. Nguồn và khả năng truy vết

Snapshot từ working tree hiện tại ngày 17.09.2026. Hash chỉ nhận diện bytes đã dùng, không tự tạo approval. PDF mới là bản đọc tham khảo; repository vẫn giữ vai trò canonical. Không thay đổi status APPROVED hoặc ngày cập nhật trong source.

PDF gốc được bảo toàn. SHA-256: 7fa362233015006d4127ec4b9da8f737e30abd318f42e02334f0e02dd9c89b07

docs/YUTA_WORKFLOW_V3.md

d6822a2b35fbc4f265b99acc94e5acf7b665f4e246a1f4f40577865d56a20b8e

docs/YUTA_AUTOMATED_CHANGE_WORKFLOW.md

373b1bd77b60c7ff4605ae2d81f609cdd1696e8be7f4b2bc1f7fc07180862d93

docs/YUTA_QA_PROTOCOL.md

f05089427b6550b64f96d6e098f362e18f5156586525f310c647c131e647aa24

docs/YUTA_KNOWLEDGE_CONSOLIDATION_PROTOCOL.md

779de9bcf2c6ee0fdf380303c96876e8dbe38fd31a7f610394dac93f80054821

docs/ui/EXTERNAL_DESIGN_INTELLIGENCE.md

1c78740b9074e0d2e7cf7c70e183ed297dd4dad5ae72b18c4ec38a612414aad4

.agents/skills/yuta-run-change/SKILL.md

17c3ac53292f81d4e45b8b3e56c7d4efcfa07a0612564194f808eed2f81311b9

.agents/skills/yuta-finish-change/SKILL.md

90522895c23e6d4e7943344e915be94cdfe15a40bc7b3951387e349e3225ce8f

.agents/skills/_engineering-primitives/AUTHORITY-PREFLIGHT.md

c073af73b14ebd21c90ed854461229a5a5c450d8d7175067530d2db81c03eed2

scripts/engineering-skills/README.md

0a2818c732b77e6f4d226fabe0c8336b0391ac68bf4b21753237e1494c876dd1

.agents/skills/yuta-research/SKILL.md

15430a2d8520839ba257afb506a207bf669fabd56eb99dc114892d08a4118fd8

.agents/skills/yuta-diagnose-bug/SKILL.md

af9dbf6f1d28a64b444803caecbfc829855b59ddbde655ffe3093d42960c71ff

.agents/skills/yuta-tdd/SKILL.md

90c10c48c47e8884abc074fb1a56a4a6050856fe50cc04ceaaa178c7f75af21e

.agents/skills/yuta-code-review/SKILL.md

a5711545dcb05480d396ee3f52f40d62339285288d497190ae4016e7eae75f2b

.agents/skills/yuta-slice-tasks/SKILL.md

fcda5a7cdc6d9f49ad0178cb6f470bcf425992d2c87c4017a9e697a5a4cc819

docs/chatGPT/YUTA_PAGE_CHAT_OPERATING_PROMPT_V3.md

fdbb2bcee502e5ee81906529db9ec2d3ff7bdf6060ec76b058c339422c3ab8d5

docs/chatGPT/YUTA_CONTROL_TOWER_OPERATING_PROMPT_V3.md

2b2c2c22e3b2a4ca7ecfd06ec2ec05242d1e6d0675b76e7177d707b81ba1db88

docs/chatGPT/YUTA_CONTROL_TOWER_HANDOFF_TEMPLATE_V3.md

ee6af141eddde233dc300e0e8fcce3ea2acb3cc40372e909179a0293164dcb52

Kiểm tra bản xuất: nội dung 18 mục lấy từ source; typography Unicode tiếng Việt, text extraction, page bounds và rendered-page review. Không tuyên bố PDF/UA hoặc screen-reader tagging.